



Ada-Context: adaptive context-aware grid-based approach for curation of data streams

Mostafa Mirzaie¹ · Behshid Behkamal¹ · Mohammad Allahbakhsh¹ ·
Samad Paydar¹ · Elisa Bertino²

Received: 25 December 2023 / Accepted: 2 March 2025 / Published online: 20 March 2025

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2025

Abstract

Stream processing and real-time applications have changed how data is collected and processed. However, data quality is crucial for its usefulness. In this paper, we introduce Ada-Context, an approach that uses external contextual information to improve data quality assessment. It involves offline and online analysis components and uses a grid structure to map streaming data to cells, enhancing performance of quality control in data streams. Results show that contextual data especially external context improves data cleansing accuracy and the grid design boosts quality control effectiveness for data streams.

Keywords Streaming data · Data quality · Contextual information · Data quality assessment · Grid-based clustering

Responsible editor: Eyke Hüllermeier.

✉ Behshid Behkamal
Behkamal@um.ac.ir

Mostafa Mirzaie
mostafa.mirzaie@mail.um.ac.ir

Mohammad Allahbakhsh
allahbakhsh@um.ac.ir

Samad Paydar
s-paydar@um.ac.ir

Elisa Bertino
bertino@cerias.purdue.edu

¹ Ferdowsi University of Mashhad, Mashhad, Iran

² Purdue University, West Lafayette, IN, USA

1 Introduction

1.1 Background

Developments in the fields of social networks, internet of things, and sensor-based technologies have resulted in the emergence of the concept of big data (Konno et al. 2018; Preuveneers and Ilie-Zudor 2019; Tang et al. 2022; Wang et al. 2022). Data streams are a form of big data, in which data is generated by diverse sources such as environment monitoring sensors, body area networks, vehicular sensors, etc., and continuously arrive at a high rate (Qian et al. 2013). Data streams have unique features such as unlimited data records, high arrival rate, and changes in data distribution (Rutkowski, Jaworski, and Duda 2020). However, due to different reasons such as malfunction, environmental noises and human interferences, these data may not have the desired quality (Aleman et al. 2018; Gupta, Bhattacharjee, and Bishnu 2021). Relying on low-quality data can result in inaccurate decisions and thus, can have serious consequences for organizations (Drouvelis and Paiardini 2022). Missed opportunities, wasted resources, lost customers, decreased customer satisfaction, and increased operational costs, are examples of negative consequences, just to name few (Rana et al. 2022)(Alzoubi, Ahmed, and Alshurideh 2022) (Abed et al. 2022). Thus, data quality assessment is vital to the success of organizations (Basu and Meckesheimer 2007). Several techniques have been proposed for assessing quality of data stream (Bhatia et al. 2022; Boniol et al. 2021; Campos et al. 2021; Chen et al. 2021; Han et al. 2022; Hu et al. 2022; Safaei et al. 2021; Yoon et al. 2021; Zhang et al. 2021). In this paper, we focus on the context-aware ones. In context-aware techniques, the data available from sources different from the source of main data stream is considered in the quality assessment process, in order to increase the accuracy of the obtained analytics (Ardagna et al. 2018). However, existing approaches suffer from two main problems. The first problem is the low accuracy of data quality assessment, which is due to the fact that these approaches only consider internal contextual information (information that exists in the same dataset) and ignore external contextual data. The second problem is the low processing performance. This is related to the adopted data processing model, which may not be efficient enough to handle the high arrival rate and unlimited data records of data streams.

1.2 Motivation scenario

Imagine a traffic monitoring system that operates in real-time and is responsible for collecting and analyzing data in order to make appropriate decisions based on the traffic conditions. The system aims to improve traffic flow, reduce congestion on the roads, and enhance the overall safety and efficiency of the transportation system based on the information provided by utilizing data from different sources situated in different geographic locations. If one of the data sources, e.g., a camera, is not working properly, it may report inaccurate data. This can lead to incorrect decisions by the monitoring system. To address such an issue, contextual information can be used to evaluate the accuracy of the data reported by the faulty data source.

For example, weather data or data reported by other cameras in nearby areas can be considered. This contextual information can help to determine whether the data reported by the faulty data source is accurate or not.

There are several types of contextual data that can be included in the traffic scenario to improve the accuracy of data quality assessment. Some examples are:

- **Weather data:** Weather conditions can have a significant impact on traffic congestion. For example, heavy rainfall or snowfall can slow down traffic, causing delays and congestion.
- **Time of day:** The time of day can also impact traffic congestion. Rush hour traffic, for example, can cause heavy congestion during certain times of the day.
- **Data from nearby data sources:** Data collected from nearby devices can be used to cross-check the data reported by the faulty data source. If the data from nearby sources does not match the data reported by the faulty data source, it can be assumed that the data reported by the faulty data source is inaccurate.
- **Historical data:** Historical data on traffic patterns and congestion in the area can also be used to evaluate the reported data. If the reported traffic conditions are significantly different from the historical patterns for that time of day, this can be an indication that the reported data are not reliable.

Therefore, contextual data can be a valuable tool for improving the accuracy of data quality assessment and cleaning. Utilizing contextual information to detect and fix low-quality data can enhance the precision of the reported data, leading to appropriate decision-making based on reliable data.

1.3 Contributions and research questions

In this research, we design a hybrid context-aware approach to address both aforementioned problems. The main research questions can be defined as follows:

- **RQ 1.** How does the integration of both internal and external contextual information improve the accuracy of quality assessment and cleaning of streaming data?
- **RQ 2.** Can a grid-based approach be developed to enhance the performance of quality control in streaming data?

To answer these questions, we propose an adaptive context-aware grid-based approach, referred to as Ada-Context, to improve the accuracy and performance of streaming data processing. It consists of two processing components – one offline and the other online. We first cluster the main data and related contextual data into a grid structure offline. This grid maps the streaming data to each cell at a higher speed by dividing the data space into cells and storing different types of metadata in each cell. Then, the data model of each cell is obtained and used in the cleansing phase. We show that this grid structure improves the assessment performance. In the online step, we use this grid and its corresponding data and models to assess quality of the data received in the online stream. This structure enables Ada-Context to

increase the accuracy without reducing the processing performance. In summary, the main contributions are as follows:

- An approach, Ada-Context, that relies on both internal and external contextual information, which improves the accuracy of the results.
- An adaptive multi-level grid-based approach for data processing to improve the performance of quality control in data streams.
- Experimental evaluation of the proposed approach using real-world datasets. The evaluation results show that our approach outperforms other similar approaches.

The rest of the paper is organized as follows. Section 2 discussed related work. Section 3 introduces our approach (Ada-Context). Section 4 presents the results of the experiments and performance comparison. Finally Sect. 5 outlines a few conclusions.

2 Related works

To better discuss the previous approaches, we first introduce five comparison dimensions. Then, we summarize and compare related approaches based on this classification.

2.1 Context type

It refers to the nature of the context that is being used to analyze or interpret the data. The different types of context commonly used in data analysis are: (i) intrinsic context—it refers to the information inherent to the data itself, such as minimum and maximum values, data types, and other characteristics of the data; (ii) systemic context—it refers to the information derived from the data source or the system that generated the data; (iii) organizational context—it refers to the information specific to the organization that is collecting or analyzing the data, such as business rules, policies, or procedures; (iv) spatiotemporal context—it refers to the information related to the location and time. This includes data related to the physical location where the data was collected, as well as the time when the data was collected.

2.2 Source type

It refers to the origin of the contextual data being used; it can be either internal or external, depending on whether the contextual data is obtained from the same dataset or other data sources. The distinction between internal and external context sources is important because it can impact the quality and relevance of the contextual data used for analysis. Internal contextual data may be more reliable and accurate, but it may also be limited in scope or variety. External contextual data may provide a broader context for analysis, but it may also introduce additional noise or errors into the analysis. For example, if social media data is being analyzed to identify trends

or sentiment, contextual data from other sources such as demographic data or news articles can be used as an external contextual information that may not be reliable.

2.3 Processing mode

The processing mode for analyzing data can be either online, offline, or hybrid, depending on how and when the actions are performed on the data. The online processing mode performs all steps of quality control in real-time. This means that the data is analyzed and cleaned as soon as it is received, allowing for immediate feedback and response to any anomalies or issues; while in the offline mode the data is first collected and stored, and then analyzed at a later time. The later approach allows for a more comprehensive view of the data, which can lead to a more precise quality assessment. Hybrid methods use a combination of both online and offline approaches to perform quality control on the data. This means that the data is first collected and stored, and then analyzed in real-time using the stored data. Hybrid methodologies balance the trade-off between runtime and precision of quality control.

2.4 Anomaly type

In data analysis, an "anomaly" refers to an unusual or unexpected data point that does not fit the expected pattern or behavior of the dataset. These anomalies can be caused by errors in data collection or recording, measurement errors, or other factors that affect the quality of the data. Three types of anomalies can occur: (i) Point anomalies occur when a single data point is significantly different from other data points in the dataset; (ii) Contextual anomalies occur when a data point is not inherently different from the rest of the data, but can be detected when the contextual information or the relationships between different attributes in the dataset are considered; (iii) Collective anomalies occur when a sequence of data points is significantly different from other sequences in the dataset.

2.5 Technique

We classify previous works into three categories of distance-based, model-based, and learning-based approaches as proposed in (Mirzaie et al. 2023).

- Distance-based techniques utilize measures of distance, such as the Euclidean distance, to quantify the difference between different data values, and identify data points with higher distance from others as potentially anomalous or low-quality data. Several approaches such as INCODE (Bharti, Pattanaik, and Pandey 2020), OFN (Gupta, Bhattacharjee, and Bishnu 2021) use weighed average and variance to identify point anomalies. Distance-based methods do not require any assumptions regarding the data distribution and can be easily applied to various data types. However, defining a suitable distance metric for large-scale datasets can be challenging.

- Model-based techniques, also known as statistical techniques, use pre-defined temporal models or set of rules to interpret contextual information and make predictions about the data. For example, the MED approach (Song et al. 2021) uses a median-based method to filter out low-quality data, while OCSVM (Wang et al. 2021) uses Kalman Filter to detect poor quality data. Model-based approaches have the advantage of simplicity, but they may struggle to adapt to new data distribution when concept drifts occur.
- Learning-based approaches use machine learning algorithms to automatically learn the patterns and relationships between data values to predict new data points. For example, CCD (Hayes and Capretz 2014) and HADF (Chen et al. 2021) detect low-quality data using regression algorithms and grid-based clustering, respectively, while Zoest's approach detect outliers in weather datasets using a classification algorithm. Model-based approaches work well in offline processing mode, but updating the model for streaming data is challenging.

We compare previous approaches with respect to five different dimensions in Table 1. As Table 1 shows, majority of the approaches heavily rely on spatiotemporal context, indicating a significant emphasis on understanding low quality data within a spatial and temporal context type. One approach uniquely considers inherent context, suggesting a departure from the typical spatiotemporal focus to potentially address anomalies inherent within the data itself. Moreover, all approaches utilize internal sources for contextual data, focusing on leveraging data within the same dataset for anomaly detection. However, this signifies a reliance on the dataset's inherent information without integrating external data sources, resulting in a reduced accuracy in detecting low-quality data. Furthermore, a few approaches ((Hayes and Capretz 2014), (Hayes and Capretz 2015), (Rassam, Maarof, and Zainal 2018)) adopt a hybrid processing mode, combining elements of both online and offline processing. This indicates a balanced approach, allowing for real-time analysis while also considering a comprehensive view of the data. Most approaches concentrate on detecting point anomalies within the dataset, where individual data points significantly differ from the rest. Fewer approaches ((Hayes and Capretz 2014), (Hayes and Capretz 2015)) partially consider contextual anomalies, focusing on anomalies detected concerning contextual information within the dataset.

This comparative analysis highlights the diverse strategies employed in detecting low-quality data streams, focusing on different contextual aspects, processing modes, anomaly types, and techniques while indicating potential areas for improvement and future research.

3 Proposed approach

In this section, we propose Ada-Context, an adaptive context-aware approach for quality control and curation of data streams. We outline the approach without delving into details in this section. However, in Sect. 4, we explain the details of our experiments as well as how we setup them, and what evaluation metrics we use for performance comparison. The overall architecture of Ada-Context is shown in

Table 1 Comparison of the state of the art

No	Context type	Source type	Processing mode	Anomaly type	Technique	Year	References
1	Spatiotemporal	Internal	Online	Point	Distance-based	2012	Zhang et al. (2012)
2	Spatiotemporal	Internal	Online	Point	Distance-based	2013	Iyer (2013)
3	Spatiotemporal	Internal	Online	Point	Distance-based	2013	Pumpichet (2013)
4	Inherent	internal	Online	Point	Distance-based	2014	Yihong Zhang, Szabo, and Sheng (2014)
5	–	–	Hybrid	Point/Contextual (partially)	Model-based	2014, 2015	Hayes and Capretz (2014), Hayes and Capretz (2015)
	Spatiotemporal	Internal	Online	Point	Learning-based		
6	Spatiotemporal	Internal	Hybrid	Point	Model-based	2018	Rassam, Maarof, and Zainal (2018)
	Spatiotemporal	Internal	Offline	Point	Learning-based	2018	Van Zoest, Stein, and Hoek (2018)
7	Spatiotemporal	Internal	Online	Point	Distance-based	2018	Aleman et al. (2018)
8	Spatiotemporal	Internal	Offline	Point	Distance-based	2020	Bharti, Pattanaik, and Pandey (2020)
9	Spatiotemporal	Internal	Online	Point	Distance-based	2021	Gupta, Bhattacharjee, and Bishnu (2021)
10	N/A	N/A	Online	Point	Learning-based	2021	Chen et al. (2021)
11	Spatiotemporal	Internal	Online	Point	Model-based	2021	Song et al. (2021)
12	N/A	N/A	Online	Point	Model-based	2021	Wang et al. (2021)
13	Time	Internal	Online	Point	Learning-based	2022	Pasini et al. (2022)
14	N/A	N/A	Online	Point	Learning-based	2022	Gao et al. (2022)
15	N/A	N/A	Offline	Point	Model-based	2023	Wang et al. (2023)

Fig. 1. Ada-Context consists of two main processing components, i.e., offline and online, each of which has some sub-components. In what follows, we discuss the framework in more details.

3.1 Offline processing

The offline processing component is mainly responsible for data preparation and building machine learning (ML) models, including three subcomponents: *preprocessing*, *data integration (Grid design)*, and *model construction*.

3.1.1 Preprocessing

In this step, both streaming data and contextual data are collected from different sources and preprocessed. The preprocessing phases are as follows:

- **Feature engineering:**
Feature engineering is the process of selecting, modifying, or creating new features (variables) from raw data to enhance the performance of machine learning models. The proposed approach encompasses both feature elimination and feature generation:
- **Feature elimination:** Removing irrelevant or redundant features from the dataset. Irrelevant features in both source and contextual data can negatively impact model performance. For instance, in the Chicago traffic dataset available on the city's data portal (used in scenario 1, Sect. 4.2), there are 22 features, many of which are redundant and can be eliminated.

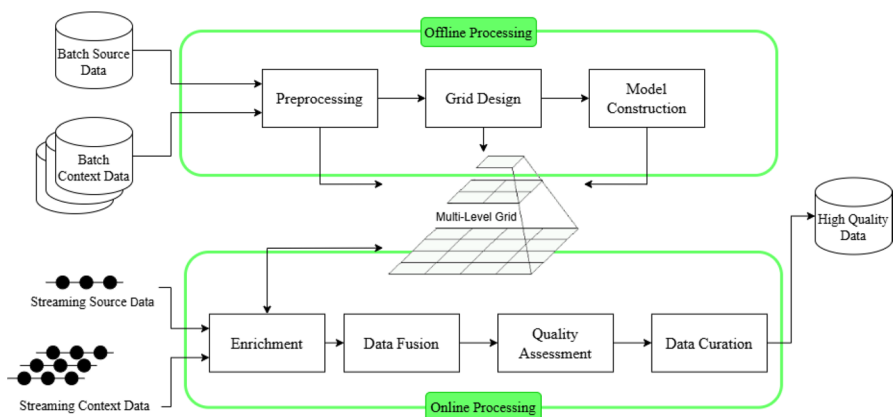


Figure 1: Ada-Context Framework

Fig. 1 Ada-Context Framework

- **Feature generation:** Creating new, informative features to better represent the data. In some cases, merging selected features can yield new insights. For example, in a traffic dataset where the number of cars in each segment is known, a new feature can be generated to represent the number of cars in neighboring segments based on their locations. Table 3 demonstrates examples of such generated features.
- **Linking through context key:** It is necessary to have a key in context-aware approaches to establish of a relationship between the source data and the contextual data, we call it: *context key*. The context key serves as a unique identifier for a particular context, which helps associate the contextual data with the source data. In other words, without a context key, we are not able to accurately link the records in the contextual data to the records in data source. Thus, the context key is essential for ensuring that the contextual data can be accurately analyzed and processed in a context-aware approach. If either the source data or some contextual data lacks a context key, they must be enriched with the appropriate feature using metadata. In the given traffic scenario, the location feature serves as context key.

3.1.2 Grid design

In this section, a multi-level grid structure is employed to organize data efficiently. The context keys determine the number of grid dimensions. If a single context key is used, the grid is one-dimensional; if two context keys are used, the grid extends to two dimensions, and so on. This approach allows for scalable and structured data representation across multiple dimensions. Consider a traffic monitoring scenario where road segment ID serves as the context key. In this case, the grid is one-dimensional, representing a continuous range of monitored locations along a road. The minimum and maximum monitored locations, denoted as X_{min} and X_{max} , define the boundaries of the road network being analyzed. When additional context keys are introduced, the grid expands into higher dimensions. For instance, if longitude and latitude are included as context keys, the grid becomes two-dimensional, segmenting the monitored area into spatial zones. This two-dimensional representation is depicted in Fig. 2-a, where the grid is structured based on two context keys, allowing traffic data to be accurately positioned.

To establish the grid, the minimum and maximum values for each context key must be identified. In the one-dimensional traffic scenario, these values are represented as X_{min} and X_{max} , where X_{min} refers to the starting location of the monitored road segment, and X_{max} represents the endpoint. For a two-dimensional grid, additional boundaries (Y_{min} , Y_{max}) are introduced. In the traffic scenario with two context keys, the grid spans from (X_{min}, Y_{min}) to (X_{max}, Y_{max}) , covering a specified geographic area. This ensures that data points are systematically partitioned into grid cells according to their spatial locations.

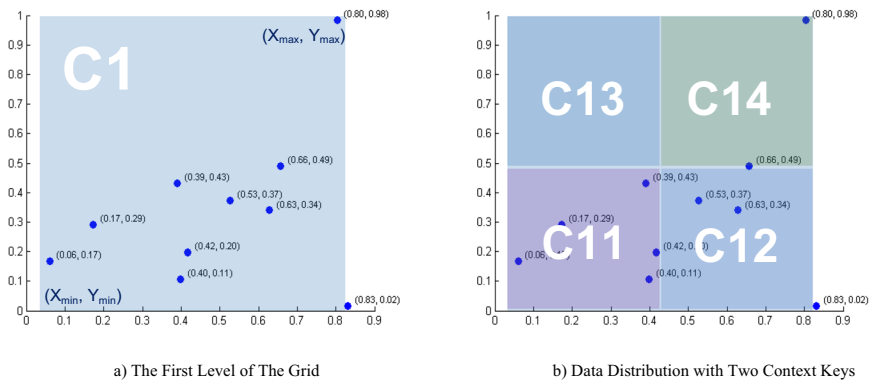


Fig. 2 Grid design with two context keys

Each grid cell is assigned a unique identifier (cId), which encodes its hierarchical level and relationship to its parent cell. The length of a cId indicates the cell's position in the hierarchy. For example:

- A cell with identifier C1 belongs to level 1.
- A sub-cell within C1, such as C14, belongs to level 2 and represents the fourth sub-cell of its parent (C1).

This hierarchical encoding facilitates efficient grid traversal and lookup operations. Figure 2-b illustrates the second level of the grid, demonstrating how data is iteratively partitioned into finer sub-cells based on traffic data distribution.

The proposed grid structure is adaptive, meaning it only subdivides cells that contain data points, ignoring empty cells. This prevents excessive computations and enhances efficiency. While there is no strict limit on the number of grid cells, selecting an optimal number improves processing speed and reduces execution time. For instance, in the traffic scenario with two context keys (longitude and latitude), each grid cell is divided into four smaller sub-cells, and the number of levels depends on the density of data. In Ada-Context, grid subdivision continues until each cell contains only a single data case, such as a surveillance camera monitoring a specific road segment.

Once the grid is established, data points sharing the same context key are grouped into the same cell, forming structured data records suitable for machine learning (ML) model training. For example, in the traffic scenario, all data within each grid cell is aggregated based on timestamps, ensuring that values from the same time range form a cohesive dataset for ML analysis. Additionally, each grid cell is annotated with metadata, including:

- Cell boundaries ($X_{\min}$, $X_{\max}$, $Y_{\min}$, $Y_{\max}$)
- Statistical summaries of source data
- Additional contextual information

This metadata annotation accelerates the online assessment step, allowing for faster assignment of new data points to their corresponding grid cells.

3.1.3 Model construction

In the last step of the offline analysis component, we build ML models based on the data stored in each cell.

There are three different types for model construction in terms of using contextual information:

- Context agnostic model (CAG): The model relies on only source data, without any contextual information.
- Semi-context aware model (SemCA): This model uses source data and internal contextual data, which are obtained from the source data.
- Context-aware model (CA): This model relies on source data as well as all available contextual information, both internal and external, where the internal and external contextual data are obtained from the source data and external data, respectively.

We need to assess which model might be beneficial within the proposed approach. However, relying solely on one model might not fulfill our requirements. For instance, in a traffic scenario where a segment lacks contextual data, employing the CA model isn't an optimal solution. Hence, our approach should have the flexibility to utilize various models in different segments, depending on the circumstances, aiming to enhance overall accuracy. Irrespective of the specific model used in the proposed approach (detailed in Sect. 4.2.5), Ada-Context undergoes training using a multi-level grid strategy. This method involves aggregating data values from the lowest cells upwards to the initial grid level, facilitating the creation of the most precise model possible.

To this end, according to Algorithm 1, it first builds the model based on the data available in the lowest level. Then, it moves up one level in the grid (to the parent cell) and builds a new model from the parent cell which is in fact the combination of data values from its all child cells. If the precision of the initial cell model is lower than the precision of the parent cell model, we take another step upwards the grandparent, and so on. This process continues until either the precision of a child cell is more than its parent, or first level of the grid is reached. Then, we select the best model, and assign it as metadata in the primary cell, to use it in the online assessment and cleansing phase. So, the Ada-Context is an adaptive approach in terms of data-aware grid partitioning as well as multi-level training that can select the best model among existing models.

Algorithm 1: Ada-Context model training.

```

Input: data values
Output: trained model
1  currentCell  $\leftarrow$  lowest level cell in the grid
2  parentCell  $\leftarrow$  parent of currentCell in the grid
3  bestModel  $\leftarrow$  null
4  bestPrecision  $\leftarrow$  0
5  while (currentCell is not the first level of the grid and precision of currentCell < precision of parentCell) do
6      model  $\leftarrow$  trainModel(data in currentCell) // Build a model based on data in the current cell
7      if (precision of model > bestPrecision) then:
8          bestModel  $\leftarrow$  model
9          bestPrecision  $\leftarrow$  precision of model
10     currentCell  $\leftarrow$  parentCell
11     parentCell  $\leftarrow$  parent of currentCell in the grid
12 End
13 if (bestModel is null) then:
14     bestModel  $\leftarrow$  trainModel(data in currentCell) // Build a model based on data in the current cell
15 return bestModel

```

Furthermore, Ada-Context employs domain adaptation as well. Domain adaptation (DA) applies an algorithm that is trained in one or more "source domains" to a different, but related "target domain" (Ben-David et al. 2010). In our approach, we use a slightly modified version of DA that uses the model of a similar cell when a data model cannot be created in another cell. In Ada-Context, this concept is modified slightly. When it is not possible to build a model for a cell, its data alongside those of similar cells are used to train a model.

3.2 Online processing

The online analysis component is responsible for using the trained models and data collected in the offline analysis component to control the quality of data arriving as a stream, in real-time. This component has four sub-components: *enrichment/integration*, *data fusion*, *quality assessment*, and *data curation*. In what follows, these sub-components are described in more details.

3.2.1 Grid-based data stream enrichment

In this step, the incoming data streams undergo enrichment to ensure they can be integrated into the established grid structure. If the data is missing a context key, it is enriched to fit within one of the grid cells. This enrichment process assigns a cell identifier (*cId*) to the data stream and links it to the appropriate trained model based on the selected grid cell.

3.2.2 Data fusion for model assignment

Once the streaming data is enriched with the context key, the next step is to map each data value to a corresponding cell in the multi-level grid, which will then be

used in the model. As previously explained, the grid partitions the data based on the context key. The process begins by comparing the context key value of the incoming data to the cell borders of the first-level cells, effectively identifying the first-level node. This comparison is then repeated with the sub-cells (children) of the selected cell, continuing until the finest level cell is found. At each step, only one child node is selected to proceed with the cell assignment process. Let C represent the total number of cells, and d denote the maximum number of children a node can have (the degree of the tree). The time complexity of the Ada-Context approach is $O(\log d(C))$, which provides an improvement over the linear time complexity found in related methods. Algorithm 2 demonstrates how the data streams are connected in our approach.

Algorithm 2: Algorithm to connect data streams.

```

Input1:   enriched streaming source data
Input2: enriched streaming context data
Output:  connected related data streams

1  rawData  $\leftarrow$  an incoming data stream of Input1
2  contextualData  $\leftarrow$  an incoming data stream of Input2 // Read External Context
3  neighbors  $\leftarrow$  neighbor id of each segment in the rawData
4  nBroadcastStream  $\leftarrow$  save neighbors as map state
5  Initialization of variables: assign zero to variable inContext, out
6  while (rawData) do // Read all enriched streaming source data
7      inContext  $\leftarrow$  find neighbor of rawData by connecting it to nBroadcastStream // Generate internal Context
8      out  $\leftarrow$  join rawData to contextualData by matching each cId
9      out  $\leftarrow$  append inContext to the out
10 End

```

Algorithm 2 describes how enriched raw data is linked with contextual data. It takes two inputs: the *rawData* (enriched streaming source data) and *contextualData* (enriched contextual data). The algorithm works by identifying neighboring segments within the raw data and storing them in a broadcast stream. It then iterates through the raw data, generating internal context by matching data points with their neighbors. The raw data is subsequently joined with the contextual data based on the context key (*cId*). Finally, the contextual information is appended to the combined data, resulting in a connected and enriched data stream that can be used for further processing or modeling.

3.2.3 Quality assessment and data curation

After mapping each connected/enriched data stream to an appropriate cell of the grid, quality assessment is executed to detect both point and contextual anomalies (Algorithm 3).

In Ada-Context, the contextual anomaly can be detected when using the SemCA and CA models. Moreover, Point anomalies can be detected using the statistical information of the historical data that is added to the grid cells as metadata in the

offline analysis component. More precisely, the streaming source data is compared to the statistical information obtained from the stored data (offline analysis component). It is considered a point anomaly. If the difference exceeds the determined threshold. To identify the contextual anomaly, the data value is compared to the value predicted by the CA model. If the difference exceeds a predefined threshold, the data value is considered to represent a contextual anomaly. In the case of detecting a low-quality data, a value predicted by the CA model or SemCA model is considered as cleansed data according to the conditions, and will replace the data value of low-quality.

Algorithm 3: Algorithm to find and clean low-quality data streams.

Input: connected data streams of source and contextual data sources
Output: cleaned data streams

```

1  input_data  $\leftarrow$  an incoming data stream of Input
2  label  $\leftarrow$  the class label feature in input_data
3  model  $\leftarrow$  grid model mentioned in input_data
4  pModel  $\leftarrow$  parent grid model mentioned in input_data
5  stats  $\leftarrow$  the statistical and historical information mentioned in input_data
6  Th  $\leftarrow$  the specified threshold to detect low-quality data
7  predicted  $\leftarrow$  the predicted value in the Input
8  parentPredicted  $\leftarrow$  the predicted value of the parent grid of each cell
9  Initialization of variables: assign correct value to variable Th
10 while (input_data) do // Read all connected data streams
11   predicted  $\leftarrow$  compute predicted based on model and input_data
12   parentPredicted  $\leftarrow$  compute parentPredicted based on pModel and input_data
13   if (label - predicted) > Th
14     if (label - parentPredicted) > Th // Check Parent Grid Contextual Data
15       if label is in stats range // Check Grid Statistical Data
16         mark count as normal data
17       else
18         mark label as anomaly data
19         clean label with predicted
20     Else
21       mark label as normal data
22   Else
23     mark label as normal data
24 End

```

Algorithm 3 outlines the process of identifying and cleansing low-quality data streams by leveraging connected traffic and contextual data. The algorithm processes incoming data streams and utilizes a grid-based model along with historical and statistical information to assess data quality. For each data point, a predicted value is computed based on the grid model, and a comparison is made against a predefined threshold Th to detect anomalies. If the difference between the actual label and the predicted value exceeds Th , the algorithm further evaluates the data using the parent grid model and statistical data to distinguish between concept drift and anomaly.

If the label falls within the expected statistical range, it is marked as normal (The occurrence of a drift); otherwise, it is flagged as an anomaly and corrected using the predicted value. This iterative process ensures that data streams are cleansed of inconsistencies, enhancing their reliability for further analysis.

As stated, the proposed approach is capable of effectively addressing the concept drift challenge. The Ada-Context provides two solutions for addressing concept drift: context-awareness and adaptability. Concept drift refers to changes in the context of streaming data and can manifest as one of the following types: sudden, incremental, recurring, gradual, or feature drift (Agrahari and Singh 2022). In most types of drift except for feature drift, the relationships between features are preserved. A context-aware approach can offer effective strategies to address this challenge. Various approaches have been proposed to address this issue, many of which share the common trait of being independent of context. Since these methods do not consider the relationships between contextual features, they require frequent model updates. In contrast, by incorporating contextual data, potential changes can be anticipated. The proposed approach is context-aware, enabling it to handle drift by tracking relevant contextual data and learning the relationships between these features.

Additionally, the proposed approach is also adaptable. If an outlier is detected at the lowest level of the grid, the method does not consider it as low-quality data but instead flags it as a potential drift. As it moves upward through the grid, the parent model is used to examine contextual changes at a higher level. If drift has occurred, it should be observable in the parent's contextual data. In the case of contextual changes, the model will detect them, and despite the data initially being flagged as an outlier at the first level, it will ultimately classify it as normal data. Therefore, the proposed approach can address various types of drift, including sudden, incremental, recurring, and gradual. If a drift occurs, it manifests in the contextual data and is managed using two strategies: context-awareness and adaptability. Furthermore, devising an effective solution to address the challenge of feature drift will be a focus of future research.

4 Experimentation and evaluation

In this section, we provide details of our experiments, including their setup and the evaluation metrics used for performance comparison. We also discuss the related works selected for comparison, along with the rationale behind their selection. We utilize Apache Flink 1.11.2 as our stream processing tool.

4.1 Evaluation metrics

In order to evaluate the performance of the proposed approach and compare it with related approaches, we need to select appropriate evaluation metrics. The selection of evaluation metrics is based on the primary objectives and research questions of the proposed approach, as outlined in the Introduction section (Sect. 1.3):

RQ 1. Improving the accuracy of quality control in streaming data by using both internal and external contextual information.

RQ 2. Enhancing the performance of quality control in data streams by employing a multi-level grid structure.

We use accuracy, precision, recall, and f-score to show the achievement of the goals as follows:

$$Accuracy = \frac{(TP + TN)}{ALL} \quad (1)$$

$$Precision = \frac{TP}{(TP + FP)} \quad (2)$$

$$Recall = \frac{TP}{(TP + FN)} \quad (3)$$

$$Fscore = \frac{2 * Precision * Recall}{(Precision + Recall)} \quad (4)$$

Since the data is time-series in nature, methods such as standard cross validation (k-fold) and bootstrap cannot be applied, as they disrupt the sequential order and relationships within the data (Bergmeir et al. 2018). For time-series data, time-series cross-validation methods, such as rolling-window cross validation or expanding-window cross validation, are typically used. In these approaches, the training set consists of observations from earlier periods, and the validation set contains data from later periods. This ensures that the temporal order is preserved, and the model is evaluated in a manner that reflects real-world forecasting (Li et al. 2014). In the proposed approach, using rolling-window cross validation, 80 percent of the dataset (from the top) is allocated to the training set, while the remaining portion is designated for the testing set.

4.2 Scenario 1: Traffic monitoring

4.2.1 Dataset

We run our experiments on real world datasets as shown in Table 2. The main dataset is the Chicago Traffic Tracker dataset (#1), which contains the source data and is enriched using the other selected datasets (#2 to #5). Data are collected from the Chicago Municipal Data Center,¹ and Time and Date² websites. In the Chicago Traffic Crashes dataset, data related to accidents at each location and time is specified. Similarly, in the Chicago Park District Event Permits dataset, events held in each

¹ <https://data.cityofchicago.org/>

² <https://www.timeanddate.com/>

Table 2 Datasets used in the proposed approach (Scenario 1)

#	Name	Number of records	Number of features	Source data/contextual data
1	Chicago traffic tracker	119 M	22	Source data
2	Chicago traffic crashes	417 K	49	Contextual data
3	Chicago park district Event permits	92.7 K	9	Contextual data (enriched by chicagoparkdistrict website)
4	Chicago weather	API	10	Contextual data
5	Chicago crimes	471 K	5	Contextual data
6	Park location	578	3	Contextual data

park are provided according to the given time range. However, this dataset lacks location information, which will be enriched during the preprocessing stage. The Chicago Weather dataset presents data on air quality and weather conditions. The Chicago Crimes dataset includes data on various types of crimes that have committed in the city, along with their corresponding locations. Finally, the Park Location dataset provides the identification number of each park along with its geographical location. The implementation codes and datasets can be accessed at the following link: https://github.com/mostafamirzaie/Traffic_Data_Quality.

4.2.2 Offline preprocessing

During the feature engineering process, relevant features are first selected as shown in Table 3. Additionally, the *weather_type* feature is derived from the *weather* feature, as outlined in the table.

Based on context-based enrichment process, geographic location (latitude, and longitude) is considered as the context key which should be observed in all selected

Table 3 Selected/generated features of dataset (Scenario 1)

#	Name	Selected features	Generated features
1	Chicago traffic tracker	time, segmentID, bus_count, message_count	–
2	Chicago traffic crashes	time, lat, long	–
3	Chicago park district event permits	start_date, end_date, parkNumber	–
4	Chicago weather	start_date, end_date, temperature, wind, humidity, barometer, visibility	weather_type
5	Chicago crimes	time, location, lat, long	–
6	Park location	parkNumber, lat, long	–
7	Chicago segments	Id, street, direction, from_street, to_street, street_headings, lat, lon	preNeighbor, nexNeighbor

datasets in Table 2. Datasets are enriched when the context key is missing. Specifically, the dataset Chicago Park District Event Permits was enriched by another dataset called Park Location due to a lack of geographic location. Also, the *weather type* is converted to a numeric format from a categorical data type. In addition to the datasets (#1 to #6), we produced a dataset from Chicago Traffic Tracker called Chicago Segments (#7). As shown, features including preNeighbor (entrance neighbor of the segment) and nexNeighbor (outgoing neighbor of the segment) are extracted by analyzing the dataset Chicago Segments and are considered as internal contextual data.

Moreover, we use correlation analysis to determine the relevant features. The number of cars in a segment is our target variable. Out of the features in our dataset, we found the following ones correlated to the target feature:

- *Weather type*: We observed almost no strong positive correlation between this feature and the number of cars, with the correlation degree of 0.022.
- *Visibility*: The visibility feature correlation is -0.7. It means that as visibility decreases, the number of cars and traffic increases.
- *Crash*: This feature has highly positive correlation (0.64) with the number of cars, meaning that the more the number of accidents, the more the number of cars and traffic will be.
- *Crime*: This feature has also a highly positive correlation (0.603) with the number of cars, which means the more the number of crime incidents, the more the number of cars and traffic will be.
- *Park events feature*: We observed that this feature has a positive correlation (0.12) with the number of cars.

4.2.3 Offline grid design based on the traffic datasets

To build the grid, the latitude and longitude are selected as two context keys. Figure 3 illustrates the first and the second levels of the generated grid. As the grid is adaptive, the cells C14 and C12 are no longer broken at level 3 due to a lack of data

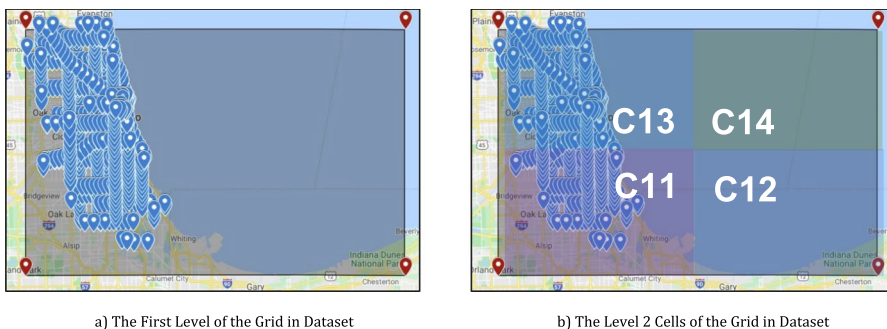


Fig. 3 Grid design in dataset traffic tracker

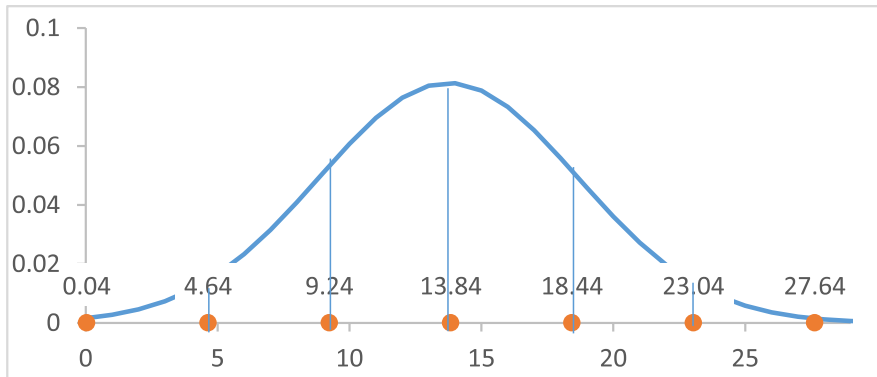


Fig. 4 Normal distribution bell curve of training dataset

case, and only cells C13 and C11 are divided. The information such as subCells (cells within a cell of the grid), border, cLevel (level of each cell in the grid), and cId as well as contextual data is added to each cell. The streaming source data and streaming context data of each cell are integrated based on the specified time range.

4.2.4 Offline model construction

After enriching the data values with cell index (cId) and integrating them, the model of each cell is built. To do so, three different models including CAg (temporal model based on the number of cars in each segment), SemCA (based on the number of cars in each segment and its preNeighbors), and CA (based on the number of cars in each segment and its preNeighbors, accidents, crimes, events, and weather) are built for each cell. Although Ada-Context possesses the capability to employ multiple models within each grid based on time intervals, this strategy is constrained in the case study due to the limited volume of data available within each grid cell, which leads to a decrease in model accuracy. Therefore, one data model in each grid cell is selected in the case study.

Research highlights the efficiency of model-based methods for streaming data due to their adaptability to evolving patterns and real-time decision-making capabilities. Models, designed for efficiency, swiftly process incoming data, predict future trends, and optimize processes, crucial for proactive decision-making in streaming data environments. These approaches can optimize resource usage, focusing on relevant data portions, reducing computational costs, and handling large volumes effectively (Mirzaie et al. 2023). Within model-based approaches, regression models offer interpretability, swift analysis, and a foundational understanding of data relationships. Their simplicity, robustness in stable patterns, and support for important features make them advantageous for rapid predictions. However, while they excel in handling less complex relationships, consideration of highly nonlinear or complex data might warrant alternative model-based approaches for superior performance (Hastie et al. 2001). Given the task of predicting a value based on multiple features,

our approach involves employing multiple regression as the preferred method among various regression techniques for constructing our models.

Since the selected model is a regression algorithm and the target variable takes continuous values, defining a threshold for outlier detection is essential. This is done using the sigma value from the normal distribution. Figure 4 illustrates the normal distribution of the count feature, representing the number of cars. In this distribution, the mean (μ) is 13.84, and the standard deviation (σ) is 4.6. Based on this, the threshold is set at 4.6, meaning any data point deviating by more than 4.6 from the predicted value is flagged as a potential outlier or concept drift, triggering further processing.

4.2.5 Performance evaluation

In this section, we evaluate the performance of our model under different setups and configurations, in order to find the optimum setup for Ada-Context. As mentioned earlier, there are two strategies to build ML models: based on the cells, or based on all grid levels. The experiments show that building models based on all levels of grid (second strategy) has an accuracy of 56.396 (training data without any contextual information) while the first strategy has an accuracy of 41.847 (more than 15% increase in accuracy). Therefore, as the evaluation results, the second strategy has been adopted for the model creation.

In the next step, we evaluate the impact of contextual data on the accuracy of the results of four variants of our proposed model that are CAg, SemCA, CA and Ada-Context. As previously indicated, the Ada-Context introduced possesses the capability to dynamically select the optimal model based on prevailing conditions, without confining itself to a singular model selection. Hence, the Ada-Context can be compared with approaches that exclusively opt for one among the CA, CAg, or SemCA models. Figure 5 shows the results of this comparison.

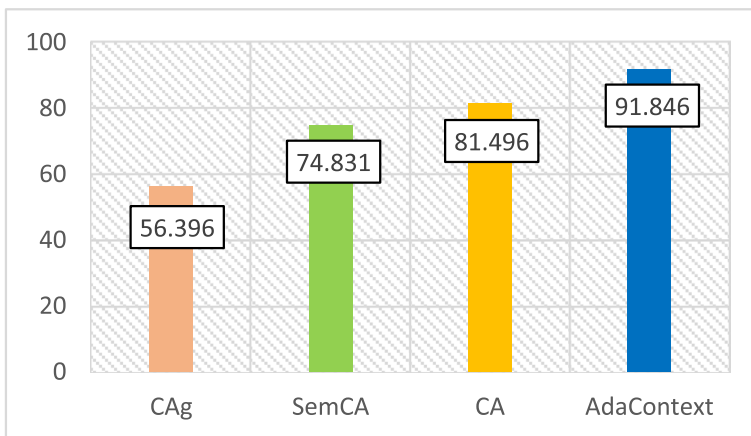


Fig. 5 Comparison of accuracy metric of CAg, CA models and Ada-Context

CA model takes advantage of both internal and external contextual data, making it more informed and better equipped to understand complex data patterns. In contrast, CAg relies solely on source data without context, while SemCA only considers internal context. As the chart shows, the CAg that is context agnostic has the lowest level of accuracy which is 56.396%; considering the internal context in SemCA increases the accuracy to 74.831%; considering both internal and external data results in the accuracy of 81.496%; and finally, considering the adaptive model boosts the accuracy to 91.846%.

Notably, 5.146% of the Ada-context accuracy (of 91.846%) stems from integrating parent cells and incorporating contextual data from neighboring elements. Therefore, the design of a multilevel grid, utilizing the domain adaptation concept explained in Sect. 0, plays a pivotal role in augmenting the model's accuracy during this phase.

Ada-Context outshines the other three models in several key aspects, including Adaptive Model Construction, and Data-Aware Partitioning. On the one hand, unlike the static models (CAg, SemCA, and CA) that rely on predefined levels of contextual information, Ada-Context dynamically constructs models at multiple levels within a grid structure. This adaptability ensures that it optimally utilizes available information, leading to higher model accuracy. On the other hand, Ada-Context's data-aware partitioning strategy allows it to partition data intelligently, ensuring that each cell receives the most relevant information. This results in more accurate models, especially in scenarios where data distribution varies across cells.

On the other hand, it is necessary to evaluate the approach in terms of runtime as well. As discussed in Sect. 0, the proposed approach, by utilizing a multi-level grid design, has effectively reduced the time complexity from exponential to logarithmic, which is well-suited to the nature of data streams. Figure 6 illustrates the time required to execute the Ada-Context approach (multi-level grid) and the CA method (without grid) after running online phase for each time window considered.

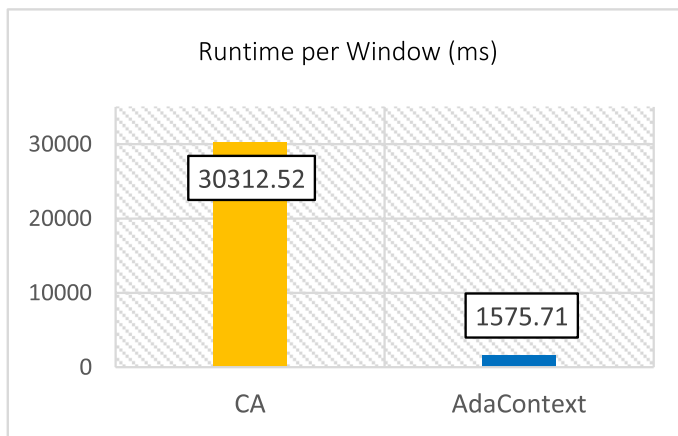


Fig. 6 Comparison of runtime metric of CA model and Ada-Context

As illustrated in the figure, the Ada-Context approach, incorporating a multi-level grid framework, achieves a significant reduction in execution time—approximately one-twentieth—compared to the CA method. This improvement highlights the dual benefits of the multi-level grid design, which not only enhances the accuracy of the quality evaluation method but also significantly decreases execution time. Given that the proposed approach is context-aware and relies on utilizing contextual data, it is essential to assess execution time in relation to the number of contextual data points. While the figure above evaluates execution time considering four external contexts, the Fig. 7 examines execution time across varying numbers of external contexts.

Apache Flink is limited to the simultaneous integration of only two data streams. Consequently, incorporating additional streams requires additional processing time, which increases approximately exponentially. In the proposed approach, utilizing four external context features results in an execution time of approximately 1.6 s. Since this process is repeated for each time window, the selection of the number of context features should align with the time window duration. In the traffic case study, the time window is set to 5 s, and four external context features are employed, ensuring that no time constraints are encountered. Moreover, employing a tool capable of integrating a greater number of data streams concurrently would further eliminate any potential time limitations.

4.2.6 Performance comparison

In the online phase, after streaming data processing by test dataset, we compared the performance of Ada-Context with seven selected related approaches, namely INCODE (Bharti, Pattanaik, and Pandey 2020), CCD (Hayes and Capretz 2014), OCPCC (Rassam, Maarof, and Zainal 2018), OFN (Gupta, Bhattacharjee, and Bishnu 2021), HADF (L. Chen et al. 2021), MED (Song et al. 2021), and OCSVM (Y. Wang et al. 2021). INCODE, and OFN are distance-based context-aware approaches, while, CCD, and OCPCC are the only two hybrid approaches that we found. HADF employs a grid clustering without using any contextual information. We compare those approaches with Ada-Context in terms of quality assessment

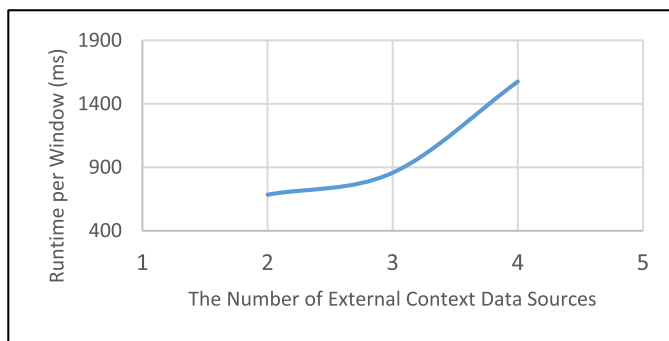


Fig. 7 Comparison of runtime metric of Ada-Context in terms of the number of the context data source

metrics such as assessment accuracy, assessment precision, assessment recall, and assessment f-score. On the other hand, we compare MED, a context-aware model-based approach, and OCSVM, a model-based approach that does not employ contextual information, with Ada-Context in terms of cleansing accuracy metric.

To conduct the comparison process, we inject poor-quality data into the test dataset. More precisely, the quality of 5% of the test data is manipulated. The poor-quality data are injected based on some ISO 25012 quality factors such as accuracy, validity, and completeness. Therefore, inaccurate (in range, while incorrect), invalid (out of range), and incomplete data are injected into the test dataset. The modified dataset is then used to check the performance of different approaches in terms of detecting and curating the poor-quality data.

Figure 8 shows the comparison results for different approaches in terms of accuracy, precision, recall, and f-score for the quality assessment stage, i.e., detecting the poor-quality data. As shown in Fig. 8-a, Ada-Context is more accurate than the other approaches. Ada-Context approach, combining internal context with

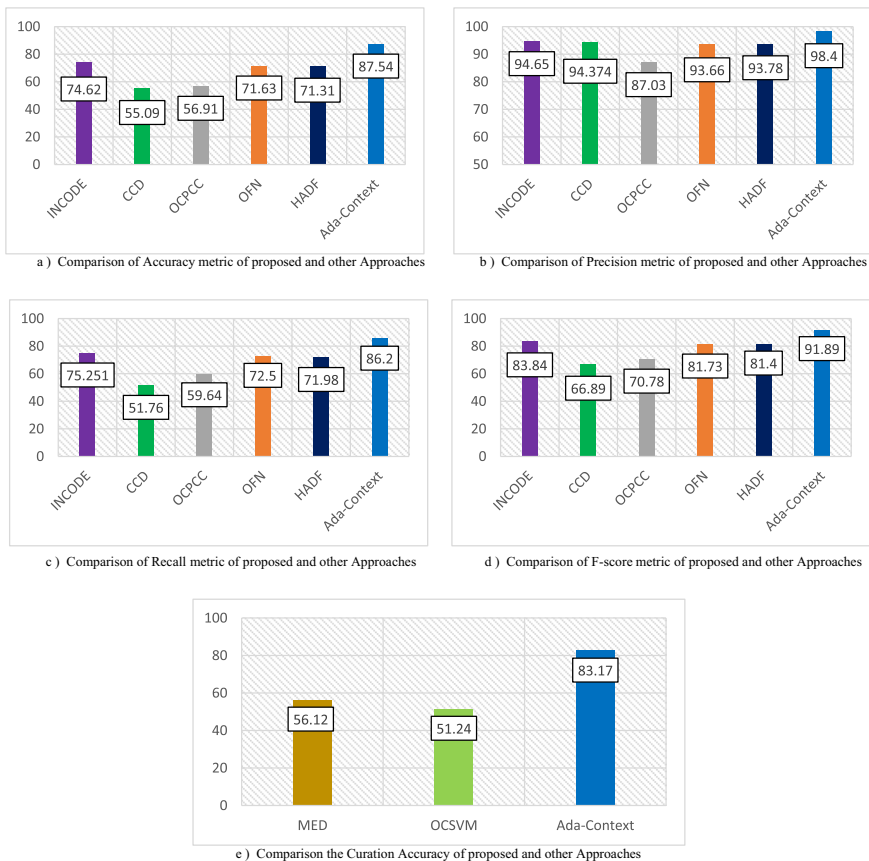


Fig. 8 Performance comparison of Ada-Context and other approaches in Scenario 1

Table 4 Datasets used in the proposed approach (Scenario 2)

#	Name	Number of records	Number of features	Source data/contextual data
1	Stroke predictions	172 K	22	Source data (contains internal contextual data)

external context, provides a more versatile and accurate solution for handling complex and dynamic data environments, making it a more effective choice compared to INCODE and other context-aware approach, which primarily relies on internal contextual information. Moreover, Ada-Context's adaptive approach, which constructs models at multiple levels within a grid, enables it to dynamically adapt to changing data patterns and complexities. In contrast, HADF's single-level grid approach lacks the adaptability to handle evolving or diverse data scenarios in one hand, and in other hand, it lacks the capability to use any contextual information. Therefore, Ada-context outperforms other context-aware and grid-based approach. Figure 8-b shows the precision related to the quality assessment. As the chart shows, Ada-Context has a higher precision than the other approached, followed by INCODE and CCD. The recall metric is another evaluated criterion, based on which the impact of false negatives is analyzed. As Fig. 8-c shows, Ada-Context exhibits a higher level of recall than the approaches due to the use of contextual information. In order to evaluate the harmonic, mean of precision and recall, we evaluated the approaches in terms of f-score (Fig. 8-d); the results shows that Ada-Context has the a highest value.

Figure 8-e shows the performance results of the approaches in terms of curating poor-quality data. As the results show, Ada-Context provides more precise curation than others by utilizing contextual and stored data. About 83% of the data, which is considered as anomalous and poor, is cleaned and correctly placed within the threshold.

By analyzing accuracy, precision, recall, and f-score metrics of Fig. 8-a to Fig. 8-e, we can answer RQ 1: *Contextual information (especially external context) can increase the accuracy in the quality assessment and cleansing of data streams.*

Moreover, by based on the results in Fig. 5 and Fig. 6, we can answer RQ 2: *The proposed approach can enhance the performance of quality control in data streams by utilizing an adaptive multi-level grid structure. This structure facilitates the selection of optimal models within each cell, thereby enhancing performance.*

4.3 Scenario 2: stroke prediction

Given that in the traffic scenario, the advantage of using external contextual data in improving the accuracy of streaming data cleansing was demonstrated, in this scenario, we intend to evaluate the performance of the proposed approach without external contextual data, relying exclusively on internal contextual data and employing the grid structure.

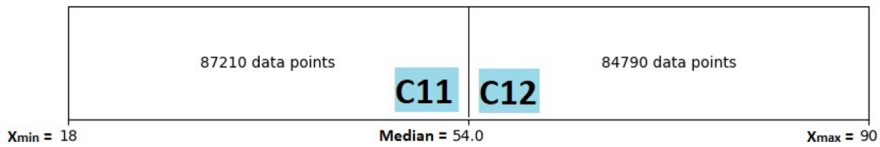


Fig. 9 The Level 2 cells of the grid in dataset stroke prediction. As shown in the figure, individuals' ages range from 18 to 90 where 18 is considered as X_{min} and 90 as X_{max} , forming the first level of the grid ($C1$). In the second level of the grid, the *median* value of age feature is calculated, and individuals with ages between X_{min} and *median* are assigned to cell *C11*, while the remaining individuals are placed in cell *C12*. Moreover, the total number of individuals in each cell is also specified. This process continues iteratively until the grid is fully constructed

4.3.1 Dataset

In Scenario 2, we conduct our experiments using an additional real-world dataset. This dataset pertains to stroke predictions among individuals of Indian origin and is available at Kaggle³ website. As shown in Table 4, the dataset comprises 172,000 records and 22 features. The *Stroke-occurrence* feature is considered as the target variable. This dataset contains multiple features that assist in diagnosing a patient's stroke, all of which will be considered as internal contextual data. The implementation code and datasets are available at the following link: https://github.com/mostafamirzaie/Stroke_Prediction.

As previously stated, in this scenario, no external contextual data will be available, and all features have been selected as internal contextual data. Therefore, the dataset serves both as a source data and as internal contextual data.

4.3.2 Offline preprocessing

In the preprocessing step, we calculated the correlation of features with the target variable. The results indicated that *alcohol consumption* and *sleep hours* features had the highest correlation with the stroke feature. Moreover, the *age* feature is designated as the context key. Since the objective is to evaluate the proposed approach without incorporating external contextual data, the dataset contains only internal contextual information.

4.3.3 Offline grid design based on the stroke dataset

To construct the grid, the *age* feature is chosen as the context key. Figure 9 depicts the second levels of the generated grid. Each cell is enriched with information such as subCells, borders, cLevel, cId and contextual data.

³ <https://www.kaggle.com/datasets/ankushpanday1/stroke-predictions-dataset-of-indians>

4.3.4 Offline model construction

Since the target variable is binary, the problem is formulated as a classification task. Unlike the previous scenario, which could be addressed using regression algorithms, this scenario necessitates the application of classification algorithms. Accordingly, *logistic regression*, *k-nearest neighbors (KNN)*, *Random Forest* and *gradient boosting* were evaluated using the entire training dataset without employing the grid structure to identify the most effective machine learning model. Notably, the proposed approach is agnostic to the choice of the machine learning algorithm, allowing for the integration of various models within the established framework.

4.3.5 Performance evaluation

In this section, various classification algorithms were evaluated to identify the most effective machine learning model for achieving more accurate predictions. The evaluation results of different classification algorithms are presented in Fig. 10.

Based on the evaluation metrics, *Gradient Boosting* outperforms the other methods across all criteria, with an accuracy of 78%, precision of 85%, and recall of 76.5%, indicating its superior ability to both accurately predict positive instances and identify the majority of true positives. *Random Forest*, with an accuracy of 72%, demonstrates similar overall performance, though it lags behind in precision and recall compared to *Gradient Boosting*. *KNN* also shows a notable improvement over *Logistic Regression*, with a precision of 75.5%. However, *Logistic Regression* achieves the lowest results across all metrics, with a F-score of 60.75%, making it the least effective model for this classification task. Therefore, *Gradient Boosting* is identified as the most robust model for this scenario. Therefore, this algorithm was selected as the final model, and the proposed approach in this scenario will utilize it within the grid structure. Therefore, in the Fig. 11, we compare the *Gradient Boosting* and *Ada-Context* approaches.

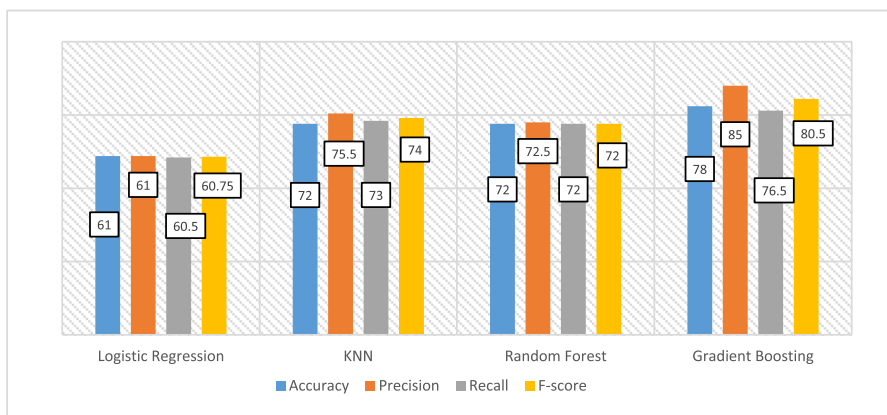


Fig. 10 Comparison of classification models

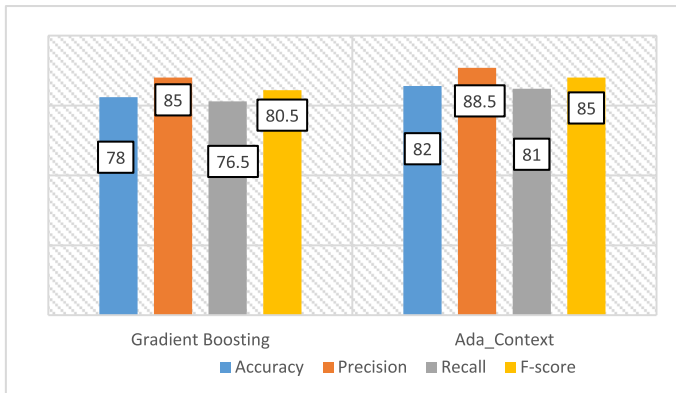


Fig. 11 Comparison of classification models

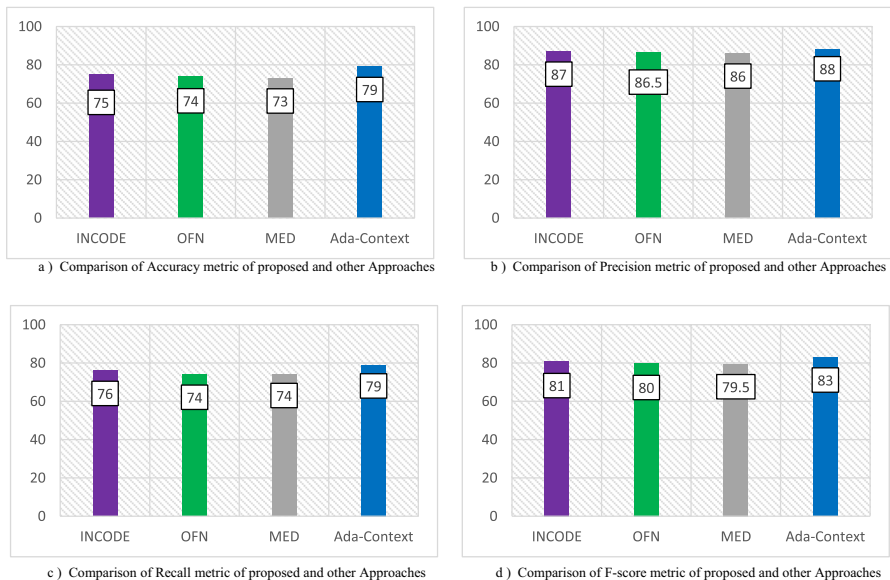


Fig. 12 Performance comparison of Ada-context and other approaches in Scenario 2

The comparison between *Gradient Boosting* and *Ada-Context* demonstrates the superior performance of *Ada-Context* across all evaluation metrics. *Ada-Context* achieves an accuracy of 82%, precision of 88.5%, recall of 81%, and F-score of 85%, outperforming *Gradient Boosting*, which attains an accuracy of 78%, precision of 85%, recall of 76.5%, and F-score of 80.5%. This indicates that *Ada-Context* not only improves the overall classification accuracy but also enhances both precision and recall, leading to more reliable and comprehensive predictions. The key advantage of *Ada-Context* lies in its utilization of a multi-level grid structure, allowing it

to adaptively select the optimal model within the grid, thereby improving classification performance.

4.3.6 Performance comparison

By executing the online phase with the test data stream, we assess our approach using evaluation metrics. Since, in the second scenario, Ada-Context utilizes internal contextual data, context-aware methods such as INCODE, OFN, and MED are selected for comparison with the proposed approach.

Figure 12 presents the comparison results of different approaches in terms of accuracy, precision, recall, and F-score. As illustrated, Ada-Context outperforms other context-aware methods. This superiority is attributed to Ada-Context's grid structure, which builds models at multiple levels within the grid.

Through the analysis of these evaluation metrics, it has been reaffirmed that using contextual information and multi-level grid structure have significantly improved the effectiveness of the proposed approach in enhancing the quality of streaming data.

5 Conclusion

This paper introduces Ada-Context, a new approach for online quality control and data stream curation. Ada-Context uses both internal and external contextual information to enhance quality control performance. It features a multi-level grid that stores different types of metadata in each cell, which improves precision. When compared to other approaches, Ada-Context proves to be effective in assessing and cleansing data streams. The grid design and data-to-cell mapping also improve performance and precision by grouping related data. As part of our future work, we plan to further explore the problem and generalize our model to work well in all situations by considering factors such as segment dispersion or low contextual data volumes.

Author contributions M.M and B.B and M.A provided the main contributions. M.M wrote the main manuscript text. B.B, M.A, S.P, and E.B revised the manuscript text. M.M and B.B provided the main figures and tables. M.A, S.P, and E.B redesigned and corrected them. All authors reviewed the manuscript.

Data availability All data of this study are available within the paper. Tables 2 and 4 describe which datasets are used in the manuscript and have cited them.

Declarations

Conflict of interest The authors declare no competing interests.

References

- Abed IA et al (2022) Creative accounting determinants and financial reporting quality: systematic literature review. *Risks* 10(4):1–25
- Agrahari S, Singh AK (2022) Concept drift detection in data stream mining : a literature review. *J King Saud Univ Comput Inform Sci* 34:9523–9540
- Aleman, Concepcion Sanchez et al. (2018) Context-Aware Data Cleaning for Mobile Wireless Sensor Networks: A Diversified Trust Approach. In: *2018 International Conference on Computing, Networking and Communications, ICNC 2018*: 226–30.
- Alzoubi HM, Ahmed G, Alshurideh M (2022) An empirical investigation into the impact of product quality dimensions on improving the order-winners and customer satisfaction. *Int J Product Quality Manage* 36(2):169
- Ardagna D, Cappiello C, Samá W, Vitali M (2018) Context-aware data quality assessment for big data. *Fut Gen Comput Syst* 89:548–562
- Basu S, Meckesheimer M (2007) Automatic outlier detection for time series: an application to sensor data. *Knowl Inf Syst* 11(2):137–154
- Ben-David S et al (2010) A theory of learning from different domains. *Mach Learn* 79(1–2):151–175
- Bergmeir C, Hyndman RJ, Koo B (2018) A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Comput Stat Data Anal* 120:70–83
- Bharti S, Pattanaik KK, Pandey A (2020) Contextual outlier detection for wireless sensor networks. *J Amb Intell Human Comput* 11(4):1511–1530
- Bhatia S et al (2022) Real-time anomaly detection in edge streams. *ACM Trans Knowl Discov Data* 16(4):1–22
- Boniol P, Paparrizos J, Palpanas T, Franklin MJ (2021) SAND: streaming subsequence anomaly detection. *Proceed VLDB Endowment* 14(10):1717–1729
- Campos D et al (2021) Unsupervised time series outlier detection with diversity-driven convolutional ensembles. *Proceed VLDB Endowment* 15(3):611–623
- Chen L, Li G, Huang G (2021) A hypergrid based adaptive learning method for detecting data faults in wireless sensor networks. *Inf Sci* 553:49–65
- Chen, Xuanhao et al. (2021) DAEMON: Unsupervised Anomaly Detection and Interpretation for Multivariate Time Series. In: *Proceedings - International Conference on Data Engineering 2021–April*: 2225–30
- Drouvelis M, Paiardini P (2022) Feedback quality and performance in organisations. *Lead Quart* 33(6):101534
- Gao H et al (2022) TSMAE: a novel anomaly detection approach for internet of things time series data using memory-augmented autoencoder. *IEEE Trans Network Sci Eng* 10:2978–2990
- Gupta U, Bhattacharjee V, Bishnu PS (2021) Outlier detection in wireless sensor networks based on neighbourhood. *Wireless Pers Commun* 116(1):443–454
- Han X, Cheng R, Ma C, Grubenmann T (2022) DeepTEA: effective and efficient online time-dependent trajectory outlier detection. *Proceed the VLDB Endowment* 15(7):1493–1505
- Hayes Michael A, and Miriam AM Capretz (2014) Contextual Anomaly Detection in Big Sensor Data. In: *2014 IEEE International Congress on Big Data*: 64–71. <http://ieeexplore.ieee.org/document/6906762/> (June 6, 2018).
- Hayes MA, Capretz MAM (2015) Contextual anomaly detection framework for big sensor data. *J Big Data* 2(1):2
- Hu Y, Ao Q, Wang Y, Work DB (2022) Streaming data preprocessing via online tensor recovery for large environmental sensor networks. *ACM Trans Knowledge Disc Data* 16:1–24
- Iyer, Vasanth. 2013. “Ensemble Stream Model for Data-Cleaning in Sensor Networks.”
- Konno H, Uchihiro N, Kosaka M (2018) Effective data cleansing method based on metadata. *Int J Japan Assoc Manage Syst* 10(1):53–58
- Li Lei, F Noorian, DJM Moss, PHW Leong (2014) Rolling Window Time Series Prediction Using MapReduce. In: *Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration, IEEE IRI 2014*: 757–64.
- Mirzaie M et al (2023) State of the art on quality control for data streams: a systematic literature review. *Comput Sci Rev* 48:100554
- Pasini K et al (2022) Contextual anomaly detection on time series: a case study of metro ridership analysis. *Neural Comput Appl* 34(2):1483–1507

- Preuveneers Davy, Elisabeth Ilie-Zudor (2019) “Special Issue: Big Data for Context-Aware Applications and Intelligent Environments.” *Future Generation Computer Systems* 99: 644–45
- Pumpichet Sitthapon (2013) “Novel online data cleaning protocols for data streams in trajectory , wireless sensor networks A Dissertation Submitted in Partial Fulfillment of the Requirements for the Degree of doctor of philosophy in electrical engineering by Sitthapo.”
- Qian Zhengping et al (2013) “TimeStream: Reliable Stream Computation in the Cloud. In: *Proceedings of the 8th ACM European Conference on Computer Systems, EuroSys 2013*: 1–14. <http://dl.acm.org/citation.cfm?id=2465351.2465353> (July 16, 2020).
- Rana NP, Chatterjee S, Dwivedi YK, Akter S (2022) Understanding dark side of artificial intelligence (AI) integrated business analytics: assessing firm’s operational inefficiency and competitiveness. *Eur J Inform Syst* 31(3):364–387
- Rassam MA, Maarof MA, Zainal A (2018) A distributed anomaly detection model for wireless sensor networks based on the one-class principal component classifier. *Int J Sens Networks* 27(3):200
- Rutkowski L, Jaworski M, Duda P (2020) “Stream Data Mining: Algorithms and Their Probabilistic Properties | SpringerLink.” <https://link.springer.com/book/10.1007%2F978-3-030-13962-9> (July 23, 2020).
- Safaei M et al (2021) Global outliers detection in wireless sensor networks: a novel approach integrating time-series analysis, entropy, and random forest-based classification. *Software Pract Exp* 52(1):277–295
- Song, Shaoxu et al. 2021. “Stream Data Cleaning under Speed and Acceleration Constraints.” *ACM Transactions on Database Systems* 46(3).
- Tang L et al (2022) Big data in forecasting research: a literature review. *Big Data Res* 27:100289
- Trevor H, Friedman J, Tibshirani R (2001) *The Elements of Statistical Learning Data Mining, Inference, and Prediction*. Springer
- van Zoest VM, Stein A, Hoek G (2018) Outlier detection in urban air quality sensor networks. *Water, Air, and Soil Pollut* 229(4):1–13
- Wang Y, Masoud N, Khojandi A (2021) Real-time sensor anomaly detection and recovery in connected automated vehicle sensors. *IEEE Trans Intell Transp Syst* 22(3):1411–1421
- Wang J, Chuqiao Xu, Zhang J, Zhong R (2022) Big data analytics for intelligent manufacturing systems: a review. *J Manufact Syst* 62:738–752
- Wang Y, Zhang R, Masoud N, Liu HX (2023) Anomaly detection and string stability analysis in connected automated vehicular platoons. *Trans Res Part c: Emerg Technol* 151:104114
- Yoon Susik, Yooju Shin, Jae Gil Lee, Byung Suk Lee (2021) Multiple Dynamic Outlier-Detection from a Data Stream by Exploiting Duality of Data and Queries. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data* (June): 2063–75.
- Zhang Y et al (2012) Statistics-based outlier detection for wireless sensor networks. *Int J Geograph Inform Sci* 26(8):1373–1392
- Zhang Y, Chen Y, Wang J, Pan Z (2021) Unsupervised deep anomaly detection for multi-sensor time-series signals. *IEEE Trans Knowledge and Data Eng.* <https://doi.org/10.1109/TKDE.2021.3102110>
- Zhang Yihong, Claudia Szabo, and QZ Sheng (2014) “Cleaning Environmental Sensing Data Streams Based on Individual Sensor Reliability.” : 405–14. http://link.springer.com/https://doi.org/10.1007/978-3-319-11746-1_29 (August 16, 2018).

Publisher’s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.